

Fakultet Tehničkih Nauka,
Novi Sad

**Predikcija konačnog odnosa strategija u Hawk-Dove
evolutivnoj igri primenom metoda mašinskog učenja**

Asistent:

Emina Demirović

Student:

Marija Milenković

RA 89/2023

Novi Sad, jun 2026. god.

Predgovor

Ova dokumentacija opisuje razvoj aplikacije i upotrebu znanja iz mašinskog učenja za predikciju ponašanja u Hawk-Dove modelu. Projekat povezuje teorijski model iz teorije igara sa praktičnim ML pristupom, gde se na osnovu parametara simulacije procenjuje konačni udeo hawk strategije u populaciji.

Cilj projekta je da omogući jednostavnu analizu različitih scenarija, kao što su promena vrednosti resursa, cena konflikta, početni odnos strategija, stopa učenja, mutacija i volatilnost okruženja. Nakon treniranja i evaluacije modela, izrađena je lokalna web aplikacija koja korisniku omogućava unos parametara i prikaz predikcije kroz pregledan interfejs.

Sadržaj

	Strana
Simulacija Hawk-Dowe igre i skup podataka	4
Eksplorativna analiza podataka	6
Pretprocesiranje podataka	8
Treniranje	9
Evaulacija	11
Pokretanje i predikcija	12
Web aplikacija	13

Uvod

Problem koji se obrađuje u ovom projektu zasniva se na Hawk-Dove modelu iz teorije igara. Ovaj model opisuje situaciju u kojoj se jedinke u populaciji takmiče za određeni resurs i mogu da biraju između dve strategije ponašanja: agresivne strategije *hawk* i mirnije strategije *dove*. Hawk strategija podrazumeva spremnost na konflikt kako bi se osvojio resurs, dok *dove* strategija izbegava direktan sukob i teži mirnijem rešavanju takmičenja.

Glavni problem je predvideti kako će se odnos ovih strategija menjati kroz vreme, odnosno koliki će biti konačni udeo hawk strategije u populaciji nakon određenog broja iteracija. Na taj ishod utiču različiti parametri, kao što su vrednost resursa, cena konflikta, početni udeo hawk jedinki, stopa učenja, stopa mutacije, volatilnost okruženja i veličina populacije.

Za rešavanje ovog problema izabrala sam regresiju, jer cilj nije bio samo da se odredi koja strategija će biti dominantna, već da se predvidi konkretna numerička vrednost: konačni udeo hawk strategije, odnosno *final_hawk*. Regresioni model je pogodan kada želimo da dobijemo kontinuiranu vrednost, na primer broj između 0 i 1, umesto samo kategorije poput “hawk” ili “dove”. Na osnovu te vrednosti kasnije se može dodatno zaključiti koja strategija je dominantna, ali osnovni zadatak ostaje predikcija procentualnog udela.

Na početku sam problem shvatila kao zadatak u kome je potrebno povezati parametre simulacije sa krajnjim ponašanjem populacije. Drugim rečima, posmatrala sam svaki scenario kao skup ulaznih vrednosti, a rezultat simulacije kao ciljnu vrednost koju model treba da nauči da predviđa. Prvi korak bio je razumevanje značenja svake promenljive i toga kako ona može da utiče na konačan odnos strategija. Na primer, veća cena konflikta može da smanji isplativost agresivnog ponašanja, dok veća vrednost resursa može da podstakne širenje *hawk* strategije.

Tokom rada pojavilo se nekoliko izazova. Jedan od glavnih izazova bio je pravilno pripremiti podatke za model, jer ulazne promenljive imaju različite opsege vrednosti. Na primer, *initial_hawk* je između 0 i 1, dok *population_size* može imati mnogo veće vrednosti. Zbog toga je bilo potrebno primeniti skaliranje kako bi podaci bili uporedivi. Drugi izazov bio je izbor odgovarajućeg modela i procena njegovog kvaliteta.

Još jedan izazov bio je tumačenje rezultata. Pošto regresioni model vraća brojčanu vrednost, bilo je potrebno povezati tu vrednost sa značenjem u okviru Hawk-Dove problema.

Na kraju, poseban deo rada bio je pravljenje jednostavne aplikacije koja omogućava da se model koristi bez direktnog rada u kodu, kroz unos parametara i prikaz rezultata u browseru.

Simulacija Hawk-Dowe igre i skup podataka

Prvi korak u projektu bio je kreiranje skupa podataka na osnovu simulacije Hawk-Dove igre. Pošto cilj projekta nije bio korišćenje unapred dostupnog dataseta, već modelovanje ponašanja populacije kroz različite scenarije, podaci su generisani pomoću skripte *main.py*.

U simulaciji se posmatra promena udela *hawk* strategije kroz više iteracija. Početna vrednost tog udela označena je promenljivom *initial_hawk*, dok se konačna vrednost nakon simulacije čuva kao *final_hawk*. Upravo *final_hawk* predstavlja ciljnu promenljivu u ovom projektu, jer model treba da nauči da predvidi koliki će biti konačni udeo *hawk* strategije za dati skup ulaznih parametara.

Skripta *main.py* sadrži funkciju *simulate_hawk_dove*, koja za jedan scenario računa kako se udeo *hawk* strategije menja kroz vreme. Na početku simulacije vrednost x predstavlja početni udeo *hawk* strategije. Zatim se kroz zadati broj iteracija računaju uspešnost *hawk* i *dove* strategije, prosečna uspešnost populacije i nova vrednost udela *hawk* strategije.

U svakoj iteraciji uzimaju se u obzir vrednost resursa V i cena konflikta C . Ove vrednosti se ne posmatraju kao potpuno fiksne, već mogu da variraju pomoću parametra *environment_volatility*. Time je simulacija realističnija, jer okruženje nije uvek stabilno.

Uspešnost *hawk* strategije računa se na osnovu toga koliko je *hawk* strategija isplativa u susretu sa drugim *hawk* ili *dove* strategijama. Ako se *hawk* susretne sa *hawk* strategijom, moguća je dobit, ali postoji i cena konflikta. Ako se *hawk* susretne sa *dove* strategijom, *hawk* dobija veću korist jer *dove* izbegava konflikt. Sa druge strane, *dove* strategija ostvaruje korist bez direktnog sukoba, ali je manje dominantna u susretu sa *hawk* strategijom.

Nakon računanja uspešnosti strategija, simulacija ažurira udeo *hawk* strategije. Na promenu utiče *learning_rate*, koji određuje koliko brzo se populacija prilagođava uspešnijoj strategiji. Dodaje se i nasumični šum, kako bi se u simulaciji predstavile slučajne promene koje se mogu javiti u realnom sistemu. Zatim se primenjuje *mutation_rate*, koji omogućava da se deo strategija promeni nezavisno od trenutne uspešnosti. Na taj način se sprečava da sistem bude potpuno deterministički.

Vrednost udela *hawk* strategije uvek se ograničava na interval od 0 do 1, jer predstavlja proporciju populacije. Nakon toga se koristi veličina populacije *population_size*, pa se broj *hawk* jedinki računa binomnom raspodelom. Time se dodatno uvodi realističnost, jer se ne posmatra samo idealna matematička proporcija, već i slučajnost koja nastaje u konačnoj populaciji.

Druga važna funkcija u fajlu je *generate_dataset*. Ona više puta pokreće simulaciju sa različitim nasumično izabranim parametrima. Za svaki scenario bira se vrednosti kao što su:

- V - vrednost resursa
- C - cena konflikta
- *initial_hawk* - početni udeo *hawk* strategije
- *iterations* - broj koraka simulacije

- population_size - veličina populacije
- learning_rate - brzina prilagođavanja strategije
- mutation_rate - stopa mutacije
- environment_volatility - promenljivost okruženja

Pored ovih osnovnih atributa, u dataset se dodaju i izvedene promenljive, kao što su *cost_value_ratio*, *value_cost_ratio*, *conflict_severity* i *theoretical_hawk*. One pomažu u boljem opisivanju odnosa između vrednosti resursa i cene konflikta. Na primer, *conflict_severity* pokazuje razliku između cene konflikta i vrednosti resursa, dok *theoretical_hawk* predstavlja teorijski očekivan udeo hawk strategije na osnovu odnosa V / C .

Na kraju, funkcija `main` generiše 10.000 scenarija i čuva ih u CSV fajl *data/raw/hawk_dove_dataset.csv*. Ovaj fajl predstavlja sirovi dataset koji se kasnije koristi u narednim fazama projekta: analizi podataka, preprocesiranju, treniranju modela i evaluaciji.

V	C	cost_value_ratio	value_cost_ratio	conflict_severity	theoretical_hawk	initial_hawk	iterations	learning_rate	mutation_rate	environment_vol	final_hawk	final_dove	hawk_dominant	dominant_strategy	population_size
15.705.164.922	13.727.474.752	8.740.738.999.2	11.440.680.245	-1.977.690.169	1.0	8.585.979.199.1	62	4.130.128.174.6	2.929.304.819.7	9.372.536.721.8	7.909.215.955.9	2.090.784.044.0	1	hawk	727
16.107.181.940	2.843.478.482.1	17.653.482.109	5.664.604.828.7	12.327.602.881	5.664.604.828.7	2.533.833.538.3	158	4.140.590.370.7	18.868.805.300	2.884.204.345.0	5.578.446.909.6	4.421.553.090.3	1	hawk	631
12.129.655.177	6.669.108.777.8	5.498.184.969.0	18.187.820.264	-5.460.546.400	1.0	6.795.270.092.0	54	4.189.883.647.1	7.002.569.629.5	5.650.074.376.9	1.0	0.0	1	hawk	292
1.003.566.229.1	27.495.937.386	27.398.229.023	36.498.709.428	-72.860.685.527	1.0	8.222.135.460.9	157	12.381.084.570	13.416.926.041	8.017.304.186.7	9.385.065.885.7	61.493.411.420	1	hawk	683
1.551.999.664.2	6.794.052.989.0	43.776.124.091	22.843.502.496	-8.725.943.652	1.0	29.231.154.342	79	11.498.213.841	18.705.138.122	4.383.913.339.6	9.322.381.930.1	6.776.180.698.1	1	hawk	974
12.057.983.446	15.528.513.119	12.878.200.728	7.765.059.895.4	34.705.296.735	7.765.059.895.4	47.214.445.102	167	11.947.775.894	1.089.997.633.1	9.806.348.292.6	7.129.186.602.8	2.870.813.397.1	1	hawk	836
4.780.177.520.2	7.547.574.660.8	15.789.318.762	6.333.395.474.8	27.673.971.405	6.333.395.474.8	6.560.476.050.6	109	1.038.029.258.2	26.817.480.075	7.628.124.420.7	6.294.416.243.6	37.055.837.563	1	hawk	394
175.822.549.102	18.492.935.141	10.517.954.173	9.507.552.357.5	9.106.802.311.5	9.507.552.357.5	17.645.765.302	194	6.069.367.841.0	29.309.825.066	13.603.295.329	9.763.333.333.3	21.666.666.666	1	hawk	600

slika 1 - izgled sirovog dataseta

Eksplorativna analiza podataka

Ova skripta služi za automatsku obradu, statističku analizu i vizuelizaciju sirovih podataka iz simulacije modela. Skripta učitava dataset (*hawk_dove_dataset.csv*), proverava tehničku ispravnost podataka (duplikate, tipove i nedostajuće vrednosti) i generiše tekstualni izveštaj (*eda_summary.txt*) sa deskriptivnom statistikom i korelacijama svih atributa sa ciljnom promenljivom (*final_hawk*). Na osnovu pokrenute skripte, dobijeni su sledeći grafički rezultati sačuvani u namenskom direktorijumu (*results/figures/eda/*):

Grafikoni *target_distribution.png* i *numeric_distributions.png* prikazuju raspodele promenljivih. Svi ulazni parametri simulacije (stope učenja, mutacije, veličina populacije, vrednost resursa V i cena C) imaju uniformnu raspodelu, što znači da su u simulacijama testirani ravnomerno i bez pristrasnosti. Nasuprot tome, ciljna promenljiva (*final_hawk*) je izrazito pomerena udesno – u najvećem broju simulacija jastrebovi na kraju postaju potpuno dominantna strategija sa udelom blizu 1.0.

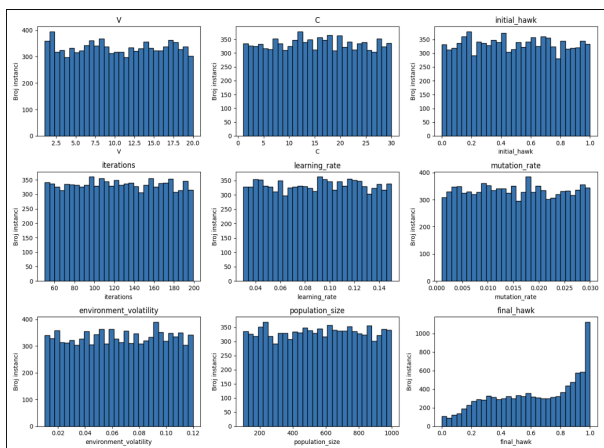
Grafikon `strategy_balance.png` prikazuje balans strategija i direktno potvrđuje dominaciju agresivnog ponašanja u sistemu. Iz rezultata se vidi da je hawk strategija krajnji pobednik u preko 6.000 simulacija, dok je dove strategija ostala dominantna u oko 3.500 simulacija.

Grafikon `boxplots.png` služi za detekciju anomalija i pokazuje da ulazni parametri nemaju ekstremnih vrednosti (autlajera). Jedini izuzetak su izvedeni odnosi `cost_value_ratio` i `value_cost_ratio`, gde se uočava dugačak niz anomalija ka gore, što je prirodna i logična posledica deljenja kada su vrednosti V ili C blizu nule.

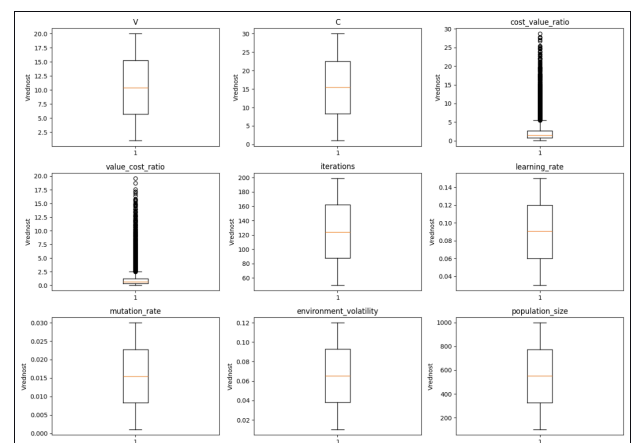
Grafikon `correlation_matrix.png` kroz toplotnu mapu korelacija pokazuje da tehnički parametri (poput stope mutacije ili veličine populacije) manje utiču na rezultat. Sa druge strane, teoretski proračun (`theoretical_hawk`) i sama vrednost resursa (V) imaju jaku pozitivnu korelaciju sa uspehom jastrebova, dok cena sukoba (C) i ozbiljnost konflikta vuku ishod u minus.

Grafikon `feature_target_relationships.png` kroz scatter prikaze otkriva ključne zavisnosti i jasne trendove u modelu. Udeo jastrebova naglo skače i stabilizuje se na maksimumu kako raste odnos vrednosti i cene (`value_cost_ratio`), dok eksponencijalno opada kada cena drastično prevaziđe vrednost resursa (`cost_value_ratio`). Sa povećanjem ozbiljnosti konflikta udeo jastrebova linearno opada. Grafikon koji poredi `theoretical_hawk` i `final_hawk` pokazuje snažno linearno poklapanje, što je ključni dokaz da se simulacija u praksi ponaša u potpunom skladu sa matematičkom teorijom igara.

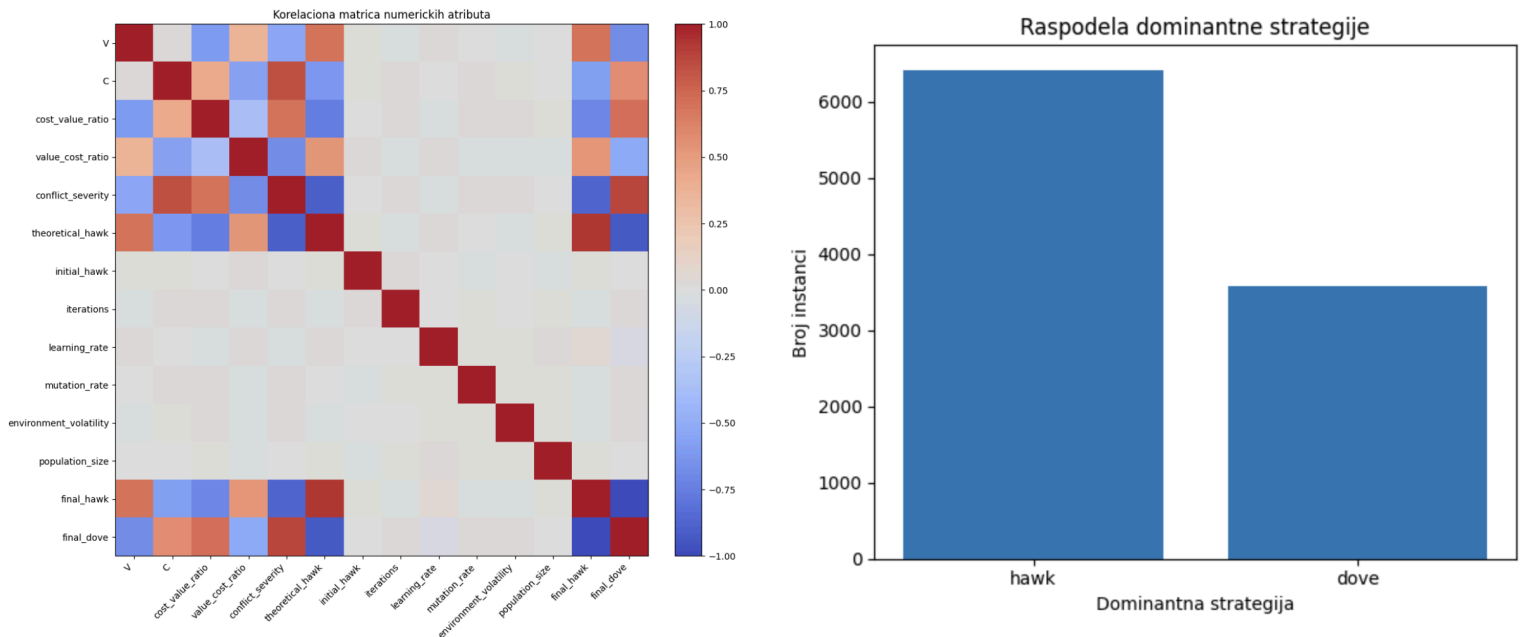
Za rad skripte korišćene su standardne Python biblioteke za naučna istraživanja i manipulaciju podacima: Pandas, NumPy, Matplotlib i Pathlib.



numeric_distribution



boxplots



Pretprocesiranje podataka

Ovaj fajl sadrži Python skriptu za pretprocesiranje sirovih podataka i njihovu pripremu za fazu treniranja modela mašinskog učenja. Skripta automatizuje proces čišćenja i podele podataka kako bi se obezbedili optimalni i stabilni uslovi za obuku modela.

Skripta prvo učitava izvorni dataset (`hawk_dove_dataset.csv`) i uklanja duplirane redove radi očuvanja integriteta podataka. Nakon toga, podaci se dele na skup nezavisnih atributa (8 ključnih sistemskih parametara: *V*, *C*, *initial_hawk*, *iterations*, *learning_rate*, *mutation_rate*, *environment_volatility* i *population_size*) i jednu ciljnu promenljivu: *final_hawk*, koja je namenjena za regresione modele.

Podela skupa podataka se vrši pomoću funkcije `train_test_split` u fiksnoj srazmeri 80:20, gde 80% podataka odlazi na trening skup, dok je 20% namenjeno za testiranje modela. Tokom ovog procesa koristi se fiksni nasumični generator (`random_state=42`) koji garantuje potpunu reproduktivnost svih rezultata pri svakom pokretanju skripte.

Nakon podele, svi pripremljeni podaci se automatski čuvaju u svom izvornom, neskalanom obliku unutar namenskog direktorijuma (`data/processed/`) u CSV formatu. Skaliranje i dalja transformacija nezavisnih atributa svesno nisu integrisani u ovu skriptu; oni će se izvršavati dinamički, unutar samog *Pipeline*-a tokom procesa treniranja, čime se na najsigurniji način sprečava curenje informacija (*data leakage*) iz test okruženja.

Za rad skripte korišćeni su standardni paketi *Pandas*, *Scikit-Learn* i *Pathlib*.

Treniranje

Ovaj fajl sadrži Python skriptu za treniranje, optimizaciju hiperparametara i evaluaciju više različitih regresionih modela nad podacima iz simulacije.

U procesu modelovanja koriste se četiri različita algoritma kako bi se testirale različite pretpostavke o podacima. Kao polazna osnova uvodi se Dummy Regressor koji uvek predviđa srednju vrednost, čime se postavlja minimalni prag uspešnosti koji svaki ozbiljniji model mora da nadmaši. Za analizu linearnih trendova koristi se Ridge regresija sa L2 regularizacijom koja stabilizuje koeficijente i sprečava preveliko prilagođavanje podacima u slučaju visoke kolinearnosti među atributima. Sličnost među scenarijima simulacije proverava se preko KNN regresije, koja donosi odluke na osnovu geografske ili metričke blizine instanci u prostoru atributa bez pretpostavke o linearnosti. Za složene, nelinearne interakcije i međusobne odnose parametara koristi se Random Forest regresor kao moćan predstavnik ansambl metoda zasnovan na strukturama stabala odlučivanja.

Svi ovi modeli su, radi očuvanja čistote koda i sprečavanja curenja podataka, spakovani u Scikit-Learn Pipeline objekte koji automatski primenjuju StandardScaler na podatke pre samog treniranja. Optimizacija hiperparametara za svaki od navedenih modela vrši se kroz praćenje kretanja RMSE vrednosti na validacionom skupu pomoću alata GridSearchCV koji automatski i objektivno kroz unakrsnu validaciju sa 5 preklopa pronalazi najbolje kombinacije na trening podacima. U poređenju sa MSE-om, RMSE se koristi zato što izvlačenjem korena vraća grešku u originalnu mernu jedinicu ciljne promenljive, čime omogućava intuitivno tumačenje stvarnog promašaja modela direktno kroz parametre simulacije. U odnosu na MAE, RMSE se preferira jer kroz kvadriranje pojedinačnih odstupanja drastično strože kažnjava velike promašaje, što je od presudnog značaja za eliminisanje rizičnih i ekstremnih grešaka u predikciji ponašanja sistema. Na kraju, RMSE je izabran umesto koeficijenta determinacije (R^2) jer meri apsolutnu grešku i ostaje stabilan i objektivno pokazatelj tačnosti tokom optimizacije, dok R^2 može veštački rasti sa povećanjem kompleksnosti modela čak i kada on počne da vrši overfitting. Kod KNN regresije, optimalan broj suseda se određuje ispitivanjem opsega vrednosti kako bi se pronašao najbolji balans između pristrasnosti i varijanse modela. Za Ridge regresiju se ispituje širok niz vrednosti parametra alfa kako bi se utvrdila optimalna jačina regularizacije. Kod Random Forest modela se pretražuje trodimenzionalna mreža parametara kroz kombinaciju broja stabala, maksimalne dubine i minimalnog broja uzoraka u listu kako bi se selektovala najuspešnija arhitektura. Tok ove optimizacije osigurava stabilnost izbora, a performanse svih kandidata se automatski vizualizuju i čuvaju u fajl *hyperparameter_cv_scores.png* u okviru rezultata.

Tokom 5-fold unakrsne validacije na kompletnom skupu podataka, Baseline Dummy Regressor je postavio minimalni prag uspešnosti sa vrednošću CV RMSE od 0.28296. Uvođenjem linearnih i prostornih pretpostavki došlo je do značajnog skoka u tačnosti. Ridge regresija je uz optimalni parametar regularizacije $\alpha=10.0$ ostvarila CV RMSE od 0.12453, dok je KNN regresor sa podešenim brojem suseda od $k=15$ pokazao još bolju prilagodljivost ostvarivši grešku od 0.11887. Najbolje performanse u ovoj fazi prikazao je Random Forest

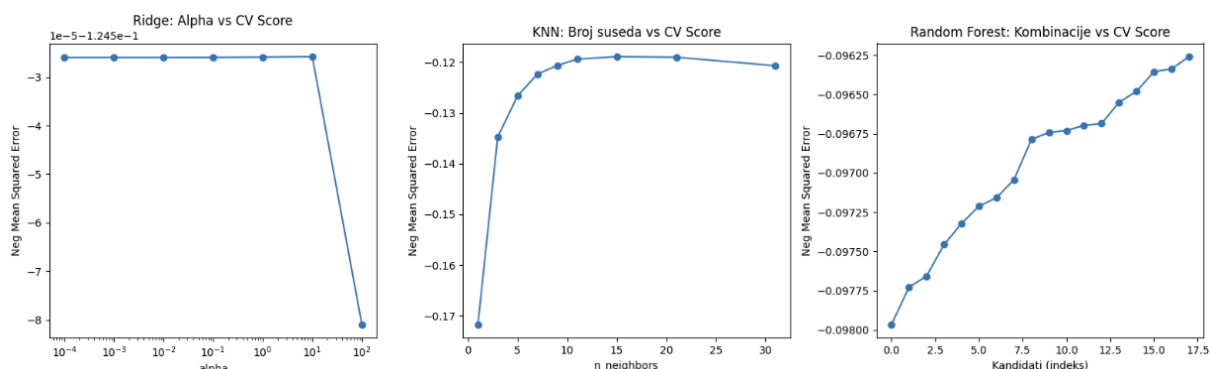
model (sa parametrima: maksimalna dubina = 8, minimalno uzoraka u listu = 5, broj stabala = 200), koji je spustio grešku na unakrsnoj validaciji na izuzetnih 0.09626.

Nakon finog podešavanja, tunirani Random Forest model se koristi za procenu značajnosti ulaznih varijabli. Atributi se rangiraju na osnovu doprinosa smanjenju varijanse unutar stabala preko ugrađene `feature_importances_` metrike, čime se jasno identifikuju najdominantniji faktori uticaja u sistemu, dok se manje bitni parametri filtriraju. Izabrana top četiri atributa se izdvajaju, a kompletan rezultat ovog koraka se trajno beleži na disku u datoteci `feature_importance.csv`.

Skripta potom vrši ponovnu optimizaciju i uporednu evaluaciju svih modela, ali ovog puta na redukovanom skupu koji sadrži samo četiri najvažnija atributa, kako bi se ispitalo da li model može biti pojednostavljen i ubrzan bez značajnog gubitka tačnosti. Nakon redukcije skupa podataka na 4 ključna atributa, sproveden je ponovni postupak unakrsne validacije. Baseline model i Ridge regresija zadržali su gotovo identične performanse (Ridge CV RMSE iznosi 0.12450), što potvrđuje da izbačeni tehnički parametri nisu nosili nikakvu linearnu informativnu vrednost. Sa druge strane, kod KNN modela došlo je do dramatičnog poboljšanja performansi gde je greška spuštена sa 0.11887 na 0.09823 uz promenu optimalnog broja suseda na $k=21$. Ovo se objašnjava činjenicom da algoritmi zasnovani na udaljenostima pate od "prokletstva dimenzionalnosti", te je uklanjanje 4 šumna i nebitna atributa drastično očistilo metrički prostor i omogućilo KNN-u da donosi znatno preciznije odluke. Pobednički Random Forest model je na redukovanom skupu neznatno popravio svoj rezultat i ostvario najniži CV RMSE u celom eksperimentu sa vrednošću od 0.09604.

Nakon što se završi unakrsna validacija na oba skupa, skripta identifikuje apsolutno najbolji model koji je ostvario najmanju grešku i testira ga na potpuno izdvojenom, skrivenom testnom skupu. Svi proračunati indikatori performansi, uključujući MAE, MSE, RMSE i R^2 , računaju se za ovaj finalni korak, a celokupan tekstualni izveštaj sa uporednim prikazom faza unakrsne validacije, izabranim parametrima i krajnjim testnim ocenama sumira se i zapisuje u izveštaj `results_summary.txt`. Na samom kraju, pobednički kompletan cevovod, koji u sebi sadrži i pripremljeni skaler i istrenirani model, serijalizuje se i trajno čuva u datoteku `best_pipeline.joblib` za kasniju direktnu primenu i predikciju na novim podacima.

Za kompletan rad ove automatizovane skripte korišćeni su standardni paketi Scikit-Learn, Pandas, Joblib i Matplotlib.



Evaulacija

Ovaj fajl sadrži Python skriptu za konačnu evaluaciju i detaljnu verifikaciju performansi sačuvanog najboljeg modela (`best_pipeline.joblib`) nad potpuno izdvojenim test skupom podataka. Glavni cilj ove skripte je da kroz napredne metričke proračune i dijagnostičke grafikone potvrdi sposobnost generalizacije modela i proveriti prisustvo eventualnih sistematskih grešaka u predviđanju.

Skripta automatski učitava test podatke (`X_test.csv` i `y_test.csv`) i primenjuje prethodno izabrani i istrenirani unifikovani cevovod. Za razliku od ručnog manipulisanja podacima, ovaj Pipeline objekat automatski u hodu skalira sirovi `X_test` kroz unutrašnji `StandardScaler` i direktno ga prosleđuje modelu na predikciju, čime se eliminiše rizik od ljudske greške. Nad generisanim predviđanjima izračunavaju se standardne i robusne metrike regresije: srednja apsolutna greška (MAE), srednja kvadratna greška (MSE), koren srednje kvadratne greške (RMSE) i koeficijent determinacije (R^2). Svi ovi tekstualni rezultati se strukturirano upisuju i trajno čuvaju na disk u fajl `evaluation_summary.txt` unutar direktorijuma `results/metrics/`.

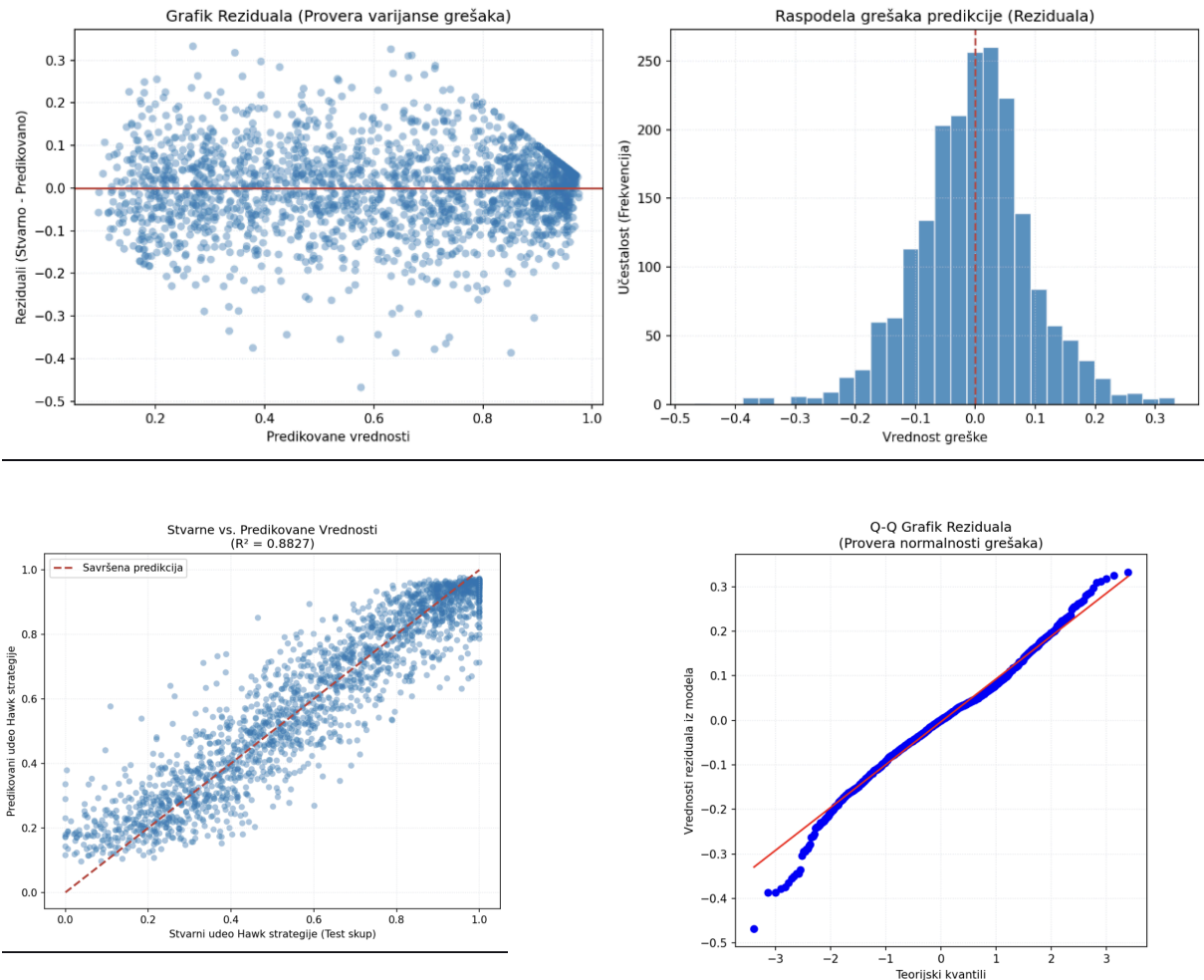
Pored statističkih pokazatelja, skripta generiše i čuva seriju od tri ključna dijagnostička fajla sa grafikonima u direktorijumu `results/figures/evaluation/` kako bi se vizuelno analiziralo ponašanje modela:

Grafikon `actual_vs_predicted.png` prikazuje odnos između stvarnih vrednosti iz simulacije (udeo Hawk strategije) i vrednosti koje je model predvideo. Tačke su koncentrisane oko idealne referentne linije savršene predikcije pod uglom od 45 stepeni, što vizuelno potvrđuje visoku vrednost koeficijenta determinacije ($\text{Test } R^2 = 0.88274$) i dokazuje da model izuzetno precizno prati stvarnu nelinearnu dinamiku populacije.

Grafikon `residual_analysis.png` sadrži dva podgrafika koji zajedno daju dubinski uvid u greške modela. Prvi podgrafik (Grafik reziduala) prikazuje greške predviđanja u zavisnosti od predikovanih vrednosti, gde ravnomerna raspodela tačaka oko nulte linije bez uočljivih levkastih oblika potvrđuje odsustvo heteroskedastičnosti i dokazuje da je model uspešno izvukao sve korisne informacije. Drugi podgrafik (Histogram raspodele grešaka) prikazuje učestalost vrednosti grešaka, potvrđujući da su reziduali centrirani tačno oko nule, što je ključni statistički dokaz da model nema sistemsku pristrasnost i da ne precenjuje niti potcenjuje ishod simulacije ptica.

Grafikon `qq_plot_residuals.png` (Q-Q grafik reziduala) predstavlja naprednu statističku proveru normalnosti grešaka. Poređenjem stvarnih reziduala iz modela sa teorijskim kvantilima normalne raspodele, ovaj grafik vizuelno i matematički potvrđuje u kojoj meri greške prate normalnu raspodu, što direktno validira pouzdanost i stabilnost donetih predikcija nad podacima iz simulacije.

Za kompletnu realizaciju procesa evaluacije, računanje metrika, statističku obradu reziduala, rukovanje putanjama i generisanje visokokvalitetnih vizuelnih prikaza korišćeni su standardni Python paketi Scikit-Learn, Pandas, NumPy, Scipy (modul stats), Joblib i Matplotlib.



Pokretanje i predikcija

Ovaj deo dokumentacije opisuje skripte koje služe za automatizaciju celokupnog procesa (pipeline) i praktičnu primenu dobijenog modela na novim, pojedinačnim primerima.

Skripta `run_pipeline.py` deluje kao centralni kontrolor celog projekta. Njen glavni zadatak je da sekvencijalno, tačnim redosledom i bez ručne intervencije pokrene sve faze razvoja modela.

Skripta `predict.py` demonstrira praktičnu primenu obučenog sistema u realnom vremenu nad pojedinačnim scenarijima. Skripta uzima rečnik sa ulaznim parametrima novog scenarija (gde su definisane specifične vrednosti za V , C , veličinu populacije, stope mutacije i učenja). Model zatim predviđa kontinuiranu vrednost za finalni udeo jastrebova (`final_hawk`), koja se programski ograničava na logički opseg između 0 i 1. Na samom kraju, skripta vrši klasifikaciju na osnovu praga od 0.5 – ukoliko je predviđeni udeo jastrebova veći od polovine, sistem ispisuje da je krajnja dominantna strategija u populaciji "hawk" (jastreb), a u suprotnom "dove" (golub).

Za realizaciju ovih zadataka korišćeni su ugrađeni Python moduli `subprocess` i `sys` za kontrolu operativnog sistema, kao i standardni paketi `Pandas`, `Joblib` i `Pathlib`.

Web aplikacija

Ovaj deo projekta pokriva proces pripreme, pakovanja i lokalnog pokretanja modela u vidu interaktivne web aplikacije.

Skripta `export_model.py` služi za eksportovanje i kreiranje konačnog paketa za produkciju (deployment bundle). Njen zadatak je da proveri postojanje istreniranog modela, kopira ih u namenski direktorijum i automatski generiše JSON fajl sa metapodacima (nazivi atributa, tip projekta, vreme kreiranja i metrički sažetak sa test skupa). Na kraju, skripta sve ove komponente kompresuje u jedinstvenu ZIP arhivu (`hawk_dove_model_bundle.zip`) spremnu za prenos i instalaciju na produkcionu server.

Skripta `app.py` predstavlja lokalni web server i aplikaciju napisanu pomoću ugrađenog Python modula `http.server`. Server učitava eksportovani model, a zatim korisniku kroz browser servira HTML korisnički interfejs sa formularom. Korisnik putem ovog interfejsa unosi vrednosti za parametre simulacije (V , C , stope učenja, mutacije itd.). Nakon klika na dugme, podaci se šalju POST metodom, aplikacija ih u pozadini skalira, prosleđuje modelu i korisniku dinamički vraća rezultat – predviđeni krajnji udeo jastrebova i podatak o tome koja je strategija dominantna u sistemu.

Za realizaciju ovih zadataka korišćeni su standardni Python moduli `http.server`, `urllib.parse`, `json`, `shutil` i `zipfile`, kao i paketi `Pandas` i `Joblib`.